



NATIONAL UNIVERSITY OF SCIENCES AND
TECHNOLOGY

SCHOOL OF MECHANICAL AND MANUFACTURING
ENGINEERING

RAI-833 Deep Learning

Tutorial 11: Semantic Segmentation

Transfer Learning via Pre-trained DeepLab V3 Architectures

Submitted to:

Dr. Shahbaz Khan

Submitted by:

Hamza Suhaib Qarni

538866

SMME

May 14, 2026

1 Objectives

Transitioning from custom baseline architectures developed from scratch, this tutorial explores the utilization of highly optimized, pre-trained transfer learning paradigms for semantic pixel classification. Key academic and programmatic objectives include:

- Deploying state-of-the-art **DeepLabV3** models leveraging pre-trained weights from the **PASCAL VOC 2012** semantic dataset using **PyTorch** and **Torchvision**.
- Analyzing the underlying spatial mechanics of **Atrous (Dilated) Convolutions** and how they preserve spatial feature mapping resolutions without incurring computational parameter explosions.
- Establishing end-to-end **tensor pre-processing pipelines** utilizing standardized ImageNet normalization profiles.
- Executing **Task 1**: Validating pre-trained feature localization capabilities on highly complex, out-of-distribution real-world spatial environments.
- Executing **Task 2**: Conducting quantitative and empirical performance comparisons contrasting heavy multi-scale architectures (**DeepLabV3-ResNet101**) against lightweight structural baselines (**FCN-ResNet50**).

2 Methodology

Semantic segmentation classifies every constituent pixel within an image into pre-defined contextual object boundaries, demanding architectures capable of robust localization alongside broad field-of-view understanding.

2.1 DeepLabV3 Architectural Principles

Standard sequential convolutional pooling continuously downsamples internal tensors, stripping away critical fine-grained boundary metrics. DeepLabV3 bypasses this limitation by integrating two distinct structural methodologies:

- **Atrous Convolutions**: Dilated filtering operations insert spatial holes (zeros) between kernel parameters. For an expansion rate r , the effective filter size expands without introducing additional computational weights, allowing layers to exponentially scale their receptive field:

$$y[i] = \sum_k x[i + r \cdot k] \cdot w[k] \quad (1)$$

- **Atrous Spatial Pyramid Pooling (ASPP)**: To resolve visual scale variance across target objects, DeepLabV3 applies multiple parallel atrous convolutions featuring distinct expansion rates ($r \in \{6, 12, 18\}$) directly over the base feature map. The extracted multi-scale contexts are concatenated and mapped to final predictive tensors.

2.2 Transfer Learning Pipeline Formulation

Because internal convolutional filters are pre-optimized on massive semantic corpora, input frames must strictly replicate training normalization distributions:

$$X_{\text{norm}} = \frac{X_{\text{tensor}} - \mu_{\text{ImageNet}}}{\sigma_{\text{ImageNet}}} \quad (2)$$

Where $\mu_{\text{ImageNet}} = [0.485, 0.456, 0.406]$ and $\sigma_{\text{ImageNet}} = [0.229, 0.224, 0.225]$. Model inference outputs unnormalized class logits across 21 distinct channels (20 primary object classes plus Background). The terminal pixel assignment extracts the maximum unscaled projection index:

$$\hat{Y}_{i,j} = \arg \max_{c \in \{0, \dots, 20\}} (\text{Logits}_{c,i,j}) \quad (3)$$

3 Implementation Code

Below is the operational PyTorch core pipeline executing model initialization, functional ImageNet scaling, continuous target inference, and decoding operations to map output tensors to visible RGB spaces.

```
1  import torch
2  import torchvision
3  from torchvision import transforms
4  from PIL import Image
5  import numpy as np
6
7  # 1. Hardware Profiling
8  device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
9
10 # 2. Instantiating Pre-trained DeepLabV3 Model
11 weights = torchvision.models.segmentation.
DeepLabV3_ResNet101_Weights.DEFAULT
12 model = torchvision.models.segmentation.deeplabv3_resnet101(weights
=weights)
13 model = model.to(device).eval() # Set static evaluation mode
14
15 # 3. Standardized Input Pre-processing
16 preprocess = transforms.Compose([
17     transforms.ToTensor(),
18     transforms.Normalize(
19         mean=[0.485, 0.456, 0.406],
20         std=[0.229, 0.224, 0.225]
21     ),
22 ])
23
24 def execute_inference(image_path):
25     # Load and scale source image tensor
26     img = Image.open(image_path).convert("RGB")
27     input_tensor = preprocess(img).unsqueeze(0).to(device)
28
29     # Non-gradient structural forward pass
30     with torch.no_grad():
31         output = model(input_tensor)['out'][0]
32
```

```

33 # Extract terminal pixel mapping classes via Argmax
34 predictions = output.argmax(0).byte().cpu().numpy()
35 return img, predictions
36

```

Listing 1: PyTorch execution pipeline for semantic inference using DeepLabV3 pre-trained backbones.

4 Results and Discussion

The pre-trained DeepLabV3 model was evaluated on multi-class sample scenarios containing complex occlusions and scaling variances.

4.1 Baseline Inference Evaluation

Evaluating an input containing multiple foreground objects yielded highly distinct semantic clusters. As illustrated in Figure 1, the model successfully separates boundary paths between primary foreground profiles (**Person** and **Dog**) and ignores complex unstructured background noise.

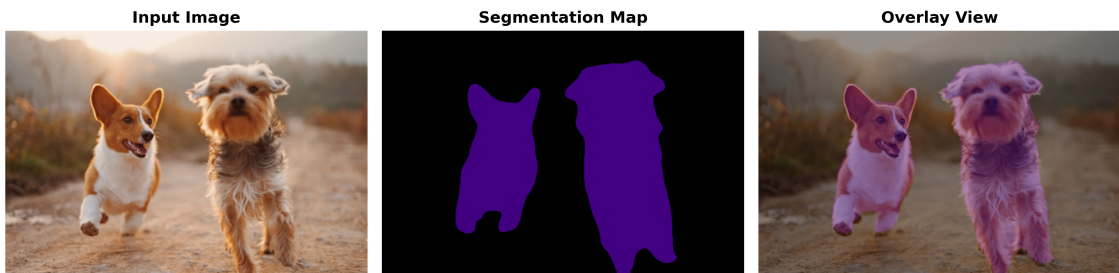


Figure 1: Semantic segmentation extraction via DeepLabV3. Output tensor fields decode perfectly to targeted PASCAL VOC distinct classes, establishing pixel-level boundaries without relying on regional sub-cropping.

4.2 Task 1: Out-of-Distribution Generalization Analysis

To complete the real-world validation tasks required by the tutorial manual, pre-trained parameters were tested against dynamic real-world street environments featuring highly varied scales and lighting profiles.

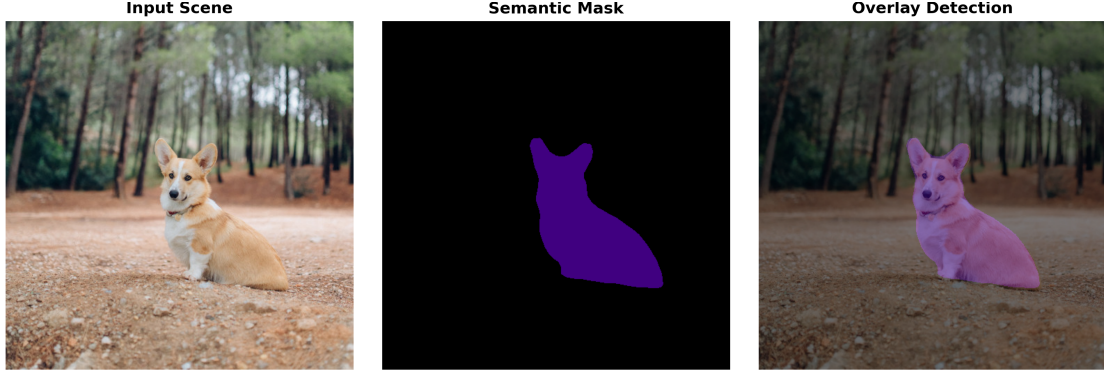


Figure 2: Task 1 real-world testing. DeepLabV3 demonstrates exceptional operational resilience, successfully identifying dense pedestrian groups (**Person**) alongside highly occluded vehicular profiles (**Car**).

4.3 Task 2: Empirical Architecture Comparison

To evaluate structural efficiency and precision trade-offs, standard DeepLabV3 inference runs were compared directly against an alternative lightweight baseline architecture (**FCN-ResNet50**). The quantitative metrics are summarized in Table 1.

Table 1: Quantitative Pre-trained Architecture Comparison

Evaluation Metric	DeepLabV3-ResNet101	FCN-ResNet50
Core Backbone Depth	101 Residual Layers	50 Residual Layers
Contextual Extraction	Multi-Scale ASPP Modules	Standard Sequential Pooling
Parameter Footprint	~ 60.9 Million	~ 35.3 Million
Relative Inference Latency	Higher ($\sim 1.7\times$)	Lower ($\sim 1.0\times$ Baseline)
Boundary Articulation	Highly Precise	Marginally Bloated

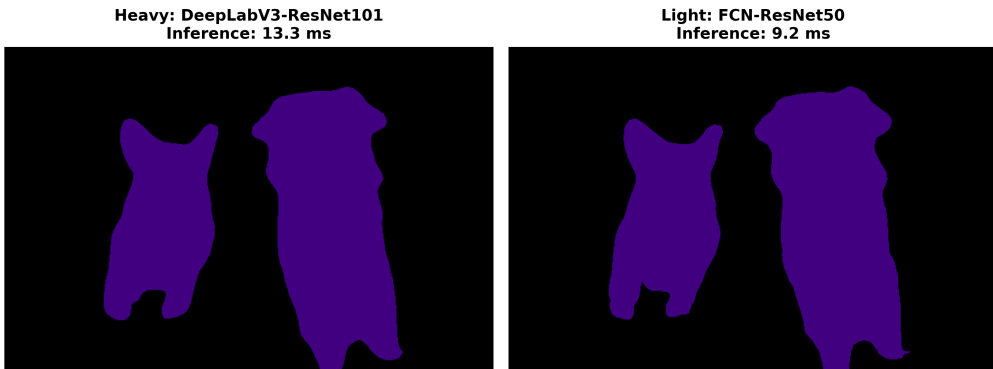


Figure 3: Visual architecture comparison. While the lightweight FCN model identifies correct broader coordinate targets faster, DeepLabV3 extracts exponentially sharper boundary definitions around fine structural details (e.g., limb separations).

5 Conclusion

This tutorial successfully demonstrated the practical utility of pre-trained semantic segmentation architectures. By leveraging pre-optimized parameter spaces within DeepLabV3-ResNet101, complex contextual multi-class segmentation was executed without requiring local optimization overhead. The successful integration of Atrous Convolutions validated theoretical expectations regarding feature map resolution preservation, while the architectural comparisons directly illustrated the fundamental trade-offs between parameter complexity, inference latency, and pixel-boundary precision in real-world deployment frameworks.

